THUNDERCLOCK · THUNDERSIM

How the simulation runs

How ThunderSim builds a simulated construction worker, makes them fall (or not), turns the motion into the four spine IMU signals, sends those through the same data path as the real device, and scores a fall model. The last part covers the models it tests.

7 November 2026 · Sources: `detection/simulation/` (README.md, MODEL_API.md, `thundersim/*.py`), `detection/models/README.md` (model card), `firmware/ORIENTATION.md`, and the research team's simulator run (RESULTS.md, SIM_SPEC.md; not in this repo)

> **Simulated data only.** The simulator checks the pipeline, thresholds, timers and failure modes before anyone wears the strip. A pass is necessary, not sufficient. It is not validated real-world fall performance, and it supports no medical claim.

What it is

Built from scratch in Python. About 5,200 lines in 17 modules under `thundersim/`, using only numpy, scipy, protobuf and matplotlib. It is not built on a physics engine, game engine or biomechanics package.

Why no physics engine. The aim is to reproduce what the four MPU-6050s and the data path would report, not to animate a body.

- Normal movement is **scripted kinematics**: smooth minimum-jerk motions with no velocity steps that could look like fake impacts.
- Falls use **small closed-form physics models**, each tested against a known result:
  - free-fall time equals $\sqrt{(2h/g)}$;
  - same-level hip speed is 3.3–4.0 m/s, against the consensus 3.4 ± 0.5 m/s;
  - harness arrest stays within OSHA limits.

Every parameter is tagged **[src]** sourced, **[design]** randomised starting value, or **[verify]** sourced but unconfirmed.

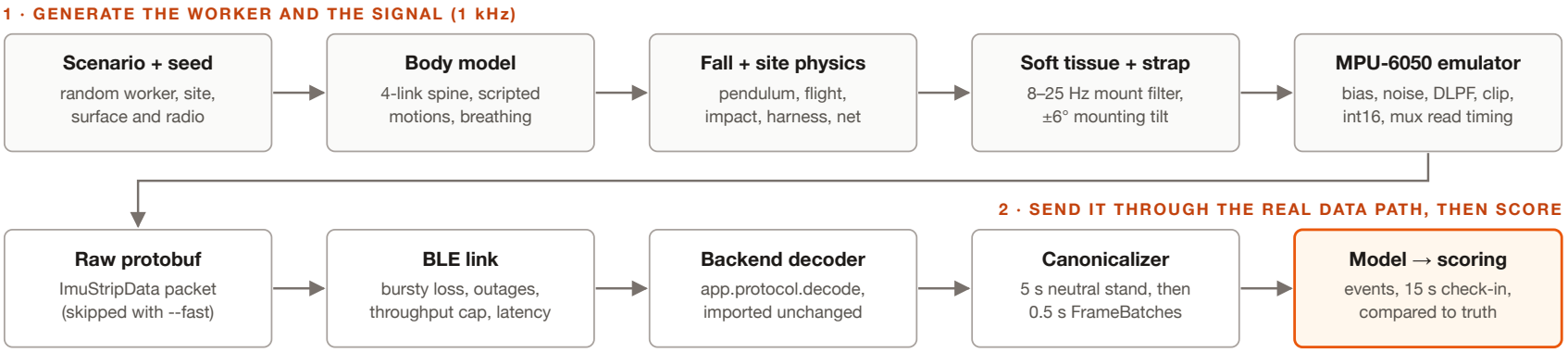
What it is for.

- Test any fall model against **35 site scenarios** under four firmware configurations.
- Count false alarms over **full fall-free 8 h shifts**.
- Find where a model breaks before anyone wears the device: dead anchors, packet loss, clipping, low sampling rates.

Size of the main run: 35 scenarios × 20 seeds × 3 firmware profiles × 2 models = **4,200 runs**. Every packet went through protobuf and the backend decoder, plus 5 fall-free 8 h shifts per model and profile.

Tests: 21 unit tests (about 15 s) cover the physics checks above, causality and the data path.

The pipeline



Each scenario also writes **ground truth**: event times such as fall onset, first impact and end of arrest, and the motion phases. Only the scoring step reads it; the model never sees it.

One run, step by step

1. **Pick a scenario and a seed.** The seed fixes every random choice, so a run can be replayed exactly. It draws:
   - **a worker:** 1.55–1.95 m tall, 55–120 kg, carrying 3–15 kg of tools and PPE;
   - **a site:** surface, wet or icy conditions, winter clothing in about 40 % of runs, temperature and radio environment;
   - **the event:** details such as fall height and direction.

   scenarios.py, site.py

2. **Script the motion.** Every run follows the same outline:
   1. A **5 s neutral stand** for calibration.
   2. A short lead-in of standing and walking.
   3. The event.
   4. **120 s of observation** afterwards.

   After a fall the worker is unconscious (40 %), still then moving (30 %) or struggling (30 %). Some scenarios have the worker get up. scenarios.py, motions.py

3. **Move the body at 1 kHz.**
   - Four rigid links: pelvis, lumbar, thoracic and upper thoracic.
   - Joints stay within soft range-of-motion limits, and voluntary trunk speed is capped at 180°/s.
   - Breathing, sway and fidgeting are always present. An unconscious worker still breathes; a strip on a table does not.
   - Each sensor sits 4 cm behind the spine, so rotation adds lever-arm acceleration.

   spine.py, motions.py

4. **Run physics only during the fall.**
   - **Loss of balance:** an inverted pendulum with knee collapse, or a foot sliding out on low friction.
   - **Flight:** ballistic. Because the body is spinning, the sensors read $\omega^2 r$ rather than a clean 0 g.
   - **Body-first impact:** a spring-damper using hip stiffness of 39–55 kN/m in series with the surface.
   - **Feet-first landing:** a short deceleration.
   - **Harness arrest:** at most 1.8 m of free fall and at most 8 kN, then hanging 15–35° forward.

   falls.py

5. **Add soft tissue and strap.** Each anchor gets a 2nd-order mount filter (8–25 Hz, softer in winter clothing) and a random mounting tilt of about ±6° (±9° in winter). scenarios.py, spine.py

6. **Emulate the MPU-6050s.** The selected firmware profile sets:
   - the sampling rate, ranges and DLPF;
   - bias, white noise and temperature-driven gyro drift;
   - int16 quantisation and clipping at full scale.

   The four sensors are read one after another through the mux, 0.8–1.5 ms apart, and share one `millis()` timestamp. device.py

7. **Packetise and transmit.**
   - Each 4-sensor scan is encoded as the firmware's raw `ImuStripData` protobuf.
   - The BLE link model adds bursty (Gilbert–Elliott) packet loss, rare 50–300 ms outages, a throughput cap and 7.5–60 ms latency.
   - The `constrained` radio (6 kB/s) shows continuous 100 Hz raw streaming overflowing the link.

   transport.py, site.py

8. **Decode with the real backend.** Packets are decoded by `backend/app/protocol.py`, imported unchanged, so a protocol bug shows up here. Rejected packets are counted rather than crashing the run. `--fast` skips this step for long runs. transport.py

9. **Canonicalise.** The 5 s neutral stand rotates each sensor so +Y points up, like the product's calibration. The stream is then cut into 0.5 s `FrameBatch` es, each holding:
   - time in seconds;
   - acceleration in m/s² and gyro in rad/s;
   - a dead-sensor mask and a clipping mask;
   - a work mode (`normal` or `lying_work`).

   canonical.py

10. **Run the model.** The simulator calls `reset(context)` once, then `update(batch)` in time order. The model returns events such as `suspected_fall`, `near_fall` or `fall_confirmed`. It must be **causal**: an event dated after its batch raises an error. model_api.py, runner.py

11. **Score against ground truth.**
    - **Run outcome:** `fall_confirmed` if any alert fired after calibration; otherwise `near_fall` if one fired; otherwise `normal`.
    - **Latency:** from the first impact (or the end of a harness arrest) to the alert reaching the laptop.
    - **Worker check-in:** every alert opens a 15 s window. A worker who is fine cancels it. An unconscious, suspended or injured worker, or a strip left on a table, does not, and the alert goes to the supervisor.

    runner.py, report.py

What gets varied

Scenario catalogue (35 scenarios)

GROUP	SCENARIOS	CORRECT OUTPUT
Falls from height	ladder, scaffold, roof edge, floor opening, excavation (1–6 m)	`fall_confirmed`
Same-level falls	slip, trip, down stairs, struck-by knock-down	`fall_confirmed`
Arrested falls	harness arrest (ARR), safety net	`fall_confirmed`
Collapse	fainting collapse standing or kneeling (no free fall)	reported only
Near-falls	stumble, rail grab, absorbed blow, slip whose outcome depends on friction	`near_fall`
Hard negatives	• **Tool work:** hammering, overhead drilling, jackhammer. • **Site work:** riding in an equipment cab, kneeling, rebar tying, ladder work, carrying, running. • **Sudden but harmless:** jumping down, sitting down hard. • **Lying down:** lying down, declared lying work, lunch nap. • **Device problems:** hard hat knocked off, strap slip, strip left on a table.	`normal`

Fall-free shifts chain 1–10 min activity blocks in SIM_SPEC 5.2's proportions for 8 h, plus about 10 jump-downs and 5 runs. They measure false alarms per shift and Stage-1 triggers per hour.

Firmware profiles

PROFILE	RATE	ACCEL	GYRO	DLPF
`current` firmware as flashed	0.5 Hz	±2 g	±250 °/s	10 Hz
`current_100hz` only the period fixed	100 Hz	±2 g	±250 °/s	10 Hz
`target` report's recommendation	100 Hz	±16 g	±2000 °/s	44 Hz
`target_8g` alternative	100 Hz	±8 g	±1000 °/s	44 Hz

Randomised per run

- **Surface:** concrete 40 %, plywood deck 25 %, packed soil 15 %, gravel 10 %, crash mat 10 %. Steel deck, scaffold plank, rebar mat and mud are modelled but not in the default mix.
- **Condition:** dry, dusty or wet, plus icy in winter. Slip scenarios use mud, ice, oil or water.
- **Radio:** open site (half of runs), steel frame or heavy equipment.
- **Device:** mux read spacing (0.8–1.5 ms) and sensor temperature, which drives gyro drift.

Honest-testing rules

- Tune only on **dev seeds 0–19**.
- Report once on held-out **test seeds 1,000,000+**.
- Give every rate a Wilson 95 % interval.
- Re-run the key scenarios with one anchor dead.

Pass bars (SIM_SPEC 5.4)

BAR	REQUIREMENT	BAR	REQUIREMENT
Falls from height confirmed	≥ 90 %	Near-falls wrongly confirmed as falls	≤ 10 %
Same-level, stairs and struck-by confirmed	≥ 80 %	Alert latency, 95th percentile	≤ 30 s
Harness arrest confirmed	≥ 80 %	False alarms in a fall-free shift	≤ 1 per 8 h

Which models it tests

The simulator accepts any class with a `reset` and an `update` method (`MODEL_API.md`). Three models have been run through it:

MODEL	WHAT IT IS	STATUS
`falldet` research-95, the main model	A trained two-stage fall detector, described below. Loaded from `detection/models`.	**TRAINED** Two versions ship: `target` (±16 g) and `_current` (±2 g, retrained).
`baseline`	The research report's hand-written two-stage rules, plus two rules the simulator suggested: "lying" must include the pelvis, and a collapse must be followed by real stillness. Thresholds were tuned on dev seeds (`tune`).	**REFERENCE** A comparison model, not the product model.
Posture estimators	Not a fall model. Segment angles come from accelerometer tilt or gyro + accelerometer fusion. Bad posture is judged by plain rules (EN 1005-4 zones, ISO 11226 holding times, side-bend, slumped curve). No machine learning or LLM is involved.	**TESTED** `posture` command; full results are in the research report *Spine posture detection results*.

`falldet`: how it decides

Input: the four anchors' acceleration and gyro, resampled to 100 Hz internally. Below **40 Hz** it reports `insufficient_rate` and detects nothing. It does not use quaternions.

Stage 1 trigger (pelvis + lumbar, small enough to port to the ESP32):

- **Free fall:** below 0.7 g for at least 100 ms.
- **Impact:** above 2.25 g.
- **Rotation:** above 3 rad/s.
- **Fires on:** an impact plus at least one other condition within 1 s, with a 5 s refractory period.
- **±2 g version:** free fall below 0.6 g and impact above 1.9 g, because ±2 g clips each axis.

Stage 2 confirmer (laptop): looks at the window from 2 s before to 8 s after the trigger. The classifier is **gradient-boosted trees** (depth 3, 200 iterations) on **28 features**:

- the free-fall dip, impact peak, jerk and rotation;
- activity before the event;
- movement after the impact, and whether walking resumed;
- each anchor's tilt, how many anchors are lying, and how still the body stays.

A numpy-only logistic-regression version is kept for C++ or JavaScript ports.

Post-rules cover cases that are not in the training data:

- **Harness arrest:** a free fall of at least 0.35 s followed by no walking is confirmed as a fall; at least 0.6 s is always confirmed.
- **Lying work:** no fall alarm if the worker declared lying work and was already lying.

Near-fall v2 (newest, added after the main simulator run) has its own trigger: rotation above 1.5 rad/s with a dip or a jolt, decided 3 s later from 22 features. Its output is a near-miss log entry, never an alarm.

`falldet`: data and lab scores

Training data (both CC BY 4.0):

- **Erciyes / UCI 455:** 17 young volunteers falling onto mats; 3,303 recordings at 25 Hz.
- **VTT-Conlot:** 13 construction workers; 3.3 h of activity.

Only the **pelvis** and **upper-thoracic** anchors are real. The lumbar and thoracic signals are **synthesised** by interpolating between them.

LEAVE-ONE-SUBJECT-OUT, 30 FOLDS (TARGET)	SCORE
Sensitivity (falls confirmed)	0.969
Specificity	0.995
F1	0.981
False alarms per hour (with the post-rules)	0.73 (0.92)
Same, pelvis-only features (F1 0.982)	0.64

- **Lab falls, not real ones.** On real falls of older adults, the best published model reached F 64.6 % (FARSEEING).
- **Four anchors not yet shown to help.** Pelvis-only scored the same as all four anchors, so any benefit of the strip still has to be shown on real 4-IMU recordings.
- **Too many false alarms for a live alarm on its own.** About 0.9 per hour is 6–7 per shift without the check-in.

The full model card is `detection/models/README.md`.

What the simulation found

TARGET FIRMWARE, `FALLDET`	DEV SEEDS	TEST SEEDS	BAR
Falls from height confirmed	98 %	100 %	≥ 90 %
Same-level, stairs, struck-by	95 %	95 %	≥ 80 %
Harness arrest confirmed	100 %	100 %	≥ 80 %
Alert latency, 95th percentile	8.3 s	8.3 s	≤ 30 s
False alarms per 8 h shift	**7.8**	–	≤ 1

The false-alarm count is for the model alone, so it is far above the bar. The 15 s check-in cancelled every one of them, and none reached a supervisor. Latency with the check-in is 23.3 s.

- **Firmware as flashed (0.5 Hz) detects nothing:** 0 of 280 falls. Fixing only the sampling period recovers almost all of them. ±2 g and the 10 Hz filter matter much less, except for the slow collapse (SYNC), which drops from 65 % to 10 %.
- **False alarms** come from hard sits (60–65 % of them confirmed as falls), equipment rides, and jump-downs followed by standing still.
- **Near-falls** were 0 % with the v1 path, which is why near-fall v2 was built. v2 scores 100 % on held-out simulator seeds, but only 59 % on real lab stumbles.
- **One dead anchor:** `falldet` keeps 97–98 % sensitivity without the pelvis or the upper-thoracic sensor.
- **Simulated falls hit harder than lab falls.** The median peak is 6.5–7 g in the simulator against 3.95 g for Erciyes mat falls. A model trained on lab falls will see larger impacts on site.

The full tables are in the research report *Fall model simulation results*.

Open item: the new agreed device spec (`firmware/ORIENTATION.md` , 7 Nov)

The team agreed on 100 Hz acquisition with ±2 g, ±250 °/s and a 10 Hz DLPF, but BLE publishes only the **latest snapshot at about 20 Hz**. None of the profiles above delivers 20 Hz.

A small probe (6 scenarios × 3 seeds) added a 20 Hz delivery profile at runtime:

- **As shipped:** `falldet` caught **0 of 12** falls, because of its 40 Hz floor.
- **Floor lowered to 15 Hz:** it matched the 100 Hz result, 10 of 12 falls.

The backend also subscribes to raw data *or* quaternions, not both, while falls need raw acceleration and the posture view needs orientation.

Next step: add an `agreed` profile, re-run the suite and shifts, and agree any floor change with research-95.

Limits

- **The body is simplified.** It is kinematic except during falls, with no arms, legs or head. Impacts use a single spring-damper contact.
- **Many values are design guesses not yet checked against real recordings,** including soft tissue, strap and radio. Measure the team's own MPU-6050s (bias, noise, read time) and strap mounting, then update `DeviceErrors` , `Worker` and `RADIO` .
- **Simulated motions are idealised.** Exported synthetic data can test and augment a model, but should never be its only training data.
- **A simulator pass is a minimum bar.** Real-world performance can only come from real recordings: mat falls, everyday activity and shadow-mode shifts.

How to run it

```
# one-time setup, from detection/simulation/
uv venv .venv --python 3.13
uv pip install --python .venv/bin/python numpy scipy "protobuf==7.36.2" matplotlib
# falldet needs these exact versions
uv pip install --python .venv/bin/python "scikit-learn==1.9.1" "joblib==1.6.0"
.venv/bin/python -m unittest discover -s tests     # 21 tests, about 15 s

# one run, with a plot of the spine and signals
.venv/bin/python -m thundersim run H-LAD --seed 3 --model falldet --profile target
--plot out.png

# main suite and fall-free shifts (dev seeds)
.venv/bin/python -m thundersim suite  --models baseline,falldet --profiles
target,current_100hz,current --runs 20 --out output/suite_main
.venv/bin/python -m thundersim shifts --models baseline,falldet --profiles
target,current_100hz,current --count 5 --hours 8 --out output/shifts_main

# held-out test seeds, once
.venv/bin/python -m thundersim suite --models falldet --profiles target --runs 10
--seed-base 1000000 --out output/test_suite
```

COMMAND	WHAT IT DOES
`list`	Scenario catalogue and device profiles
`run`	One scenario and seed: prints ground truth and model events, draws a plot
`suite`	Randomised batch; writes `runs.jsonl`, `report.md`, `summary.json`. Add `--dead-slot` to kill an anchor, or `--fast` to skip protobuf.
`shifts`	Fall-free 8 h shifts: false alarms and Stage-1 triggers
`tune`	Random search of the baseline's thresholds on dev seeds
`export`	Labelled synthetic dataset, one `.npz` per run
`calibrate`	Compares simulated mat falls with Erciyes impact peaks
`posture`	Spine posture accuracy and bad-posture rules
`report`	Merges several output folders into one report

The simulator code is in `detection/simulation/thundersim/` and the fall model in `detection/models/`. Plug in your own model with `--models package.module:ClassName`.